



거울아 거울아 지금 내 앞에 있는 것이 무엇이더냐

GitHub Copilot · MewUI · MCP로 데스크톱 바이브 코딩의 실체를 비춰 보기

남정현 · 닷넷데브 (.NET Dev) · Microsoft MVP (Dev&Tech)

GitHub Copilot Dev Days

SEOUL April 3



by .NET Dev



이 세션에서 다룰 내용

이번 세션에서는 '거울'이라는 개념을 중심으로 데스크톱 바이브 코딩의 핵심 요소들을 탐색합니다.

01

거울의 개념과 흔한 오해

데스크톱 바이브 코딩의 본질과 이에 대한 일반적인 오해들을 살펴봅니다.

02

MewUI와 .NET 10의 중요성

왜 MewUI와 .NET 10이 바이브 코딩에서 핵심적인 역할을 하는지 알아봅니다.

03

거울 프레임워크의 구조

거울 프레임워크의 내부 작동 방식과 구조를 이해합니다.

04

거울과 효과적으로 대화하는 방법

거울에게 잘 통하는 프롬프트와 질문 유형을 통해 효율적인 활용법을 배웁니다.

05

실제 적용 및 한계점

실제 앱 개발 사례를 통해 적용 방안을 제시하고, 거울의 한계점도 명확히 인지합니다.

거울 앞에서 가장 많이 하는 착각

착각 ①

프롬프트만 영리하면 데스크톱 앱도 술술 나올 것이다 — 실제로는 맥락과 제약이 없으면 Copilot은 방향을 잃습니다.

착각 ②

웹에서 먹히던 감각이 데스크톱에서도 그대로 통할 것이다 — 플랫폼 API와 UX 모델은 전혀 다른 세계입니다.

착각 ③

코드가 생성되면 앱도 만들어진 것이라고 볼 수 있다 — 빌드와 실행, 그리고 사람의 눈으로 확인해야 비로소 앱이 됩니다.



핵심 명제

원본 소스 없이도 거울은 작동한다

소스 없는 세계의 현실

컴파일된 바이너리만 존재하는 환경에서는 Copilot이 API 시그니처, 타입 정보, 메서드 목록을 추측에 의존할 수밖에 없습니다. 추측은 곧 오류이고, 오류는 빌드 실패로 이어집니다.

리플렉션 MCP가 채우는 공백

리플렉션 기반 MCP 서버는 런타임에 어셈블리를 탐색해 실제 타입·멤버 정보를 Copilot에게 실시간으로 제공합니다. 소스 코드 없이도 정확한 API 컨텍스트를 확보할 수 있습니다.

리플렉션 MCP는 Copilot의 눈이 닿지 않는 곳에 거울을 놓는 일입니다.

AI 에이전트는 사실 눈 뜬 장님입니다.

MirrorMirror MCP 서버 없이 AI 에이전트가 MewUI를 만날 때, 예측 가능한 붕괴 시나리오가 펼쳐집니다.

01

WPF 중심의 추측으로 시작

훈련 데이터에 없는 MewUI 앞에서 에이전트는 자신이 아는 최선인 WPF 패턴으로 코드를 생성합니다. 처음엔 그럴듯해 보입니다.

02

첫 빌드 오류 — GitHub 리포지터리 탐색

예상대로 빌드가 실패합니다. 에이전트는 GitHub 리포지터리를 찾아 README와 샘플 코드를 읽으려 시도합니다.

03

해결 안 되면 — 인터넷 검색으로 전환

리포지터리만으로 부족하면 웹 검색으로 넘어갑니다. 2026년 1월 이후의 신생 프레임워크이므로 유의미한 결과가 거의 없습니다.

04

MewUI를 '오류 투성이 라이브러리'로 평가 절하

검색으로도 해결되지 않으면 에이전트는 MewUI 자체에 결함이 있다고 판단하기 시작합니다. "이 라이브러리는 불안정합니다"라는 결론을 내립니다.

05

제거 또는 포기 — 파괴적 결론

최종적으로 MewUI를 다른 프레임워크로 교체하거나 작업 자체를 포기하는 방향을 제안합니다. 문제는 라이브러리가 아니라 컨텍스트의 부재였는데도.

❏ 이 시나리오는 에이전트의 실패가 아닙니다. 거울을 주지 않은 환경의 실패입니다.



MewUI와 .NET 10을 고른 이유

- MewUI는 2026년 1월 6일 첫 커밋이 올라온 신생 오픈 소스 프레임워크입니다. 훈련 데이터에 포함될 수 없는 코드베이스 — 이것이 이번 세션에서 MewUI를 선택한 핵심 이유입니다.



훈련 데이터 밖의 프레임워크

2026년 1월에 처음 공개된 MewUI는 어떤 AI 모델의 훈련 데이터에도 포함되지 않습니다. Copilot이 추측 없이 코드를 생성하려면 리플렉션 기반 MCP 같은 실시간 컨텍스트 공급이 필수입니다.



app.cs 단일 파일 + 명시적 의존성

한 파일로 시작하고 의존성이 코드 상단에 선언되므로, Copilot이 프로젝트 맥락을 정확히 파악할 수 있습니다.



크로스플랫폼 + NativeAOT

Windows·Linux·macOS를 단일 코드베이스로 지원하며, NativeAOT로 런타임 없이 배포까지 하나의 흐름으로 연결됩니다.
(MIT 라이선스 오픈 소스)



짧은 반복 루프

생각 → 생성 → 검증 사이클이 짧아 바이트 코딩의 흐름을 끊지 않고 빠르게 이터레이션할 수 있습니다.

제가 제안하는 MewUI 바이브코딩 프레임워크의 구성

이 작업 공간 템플릿은 샘플 코드보다 '생성 규칙'에 무게를 둡니다. 각 구성 요소가 맡은 역할을 명확히 분리합니다.

구성 요소	역할
copilot-instructions.md	Copilot 규칙·금지 사항·검증 순서 정의 — Copilot이 가장 먼저 읽는 헌법
mcp.json / mcp-config.json	VS Code와 CLI에서 동일한 MCP 도구 세트를 바라보도록 설정
artifacts/	빌드와 배포 결과물을 모으는 기본 위치 — 산출물 경로를 통일
거의 비어 있는 루트	불필요한 샘플을 제거해 Copilot의 노이즈를 최소화

이 프레임워크를 적용했을 때의 흐름

1

① 명령 지시

어떤 코드를 써야 하고 무엇을 하면 안 되는지 제한합니다. `copilot-instructions.md`가 핵심입니다.

2

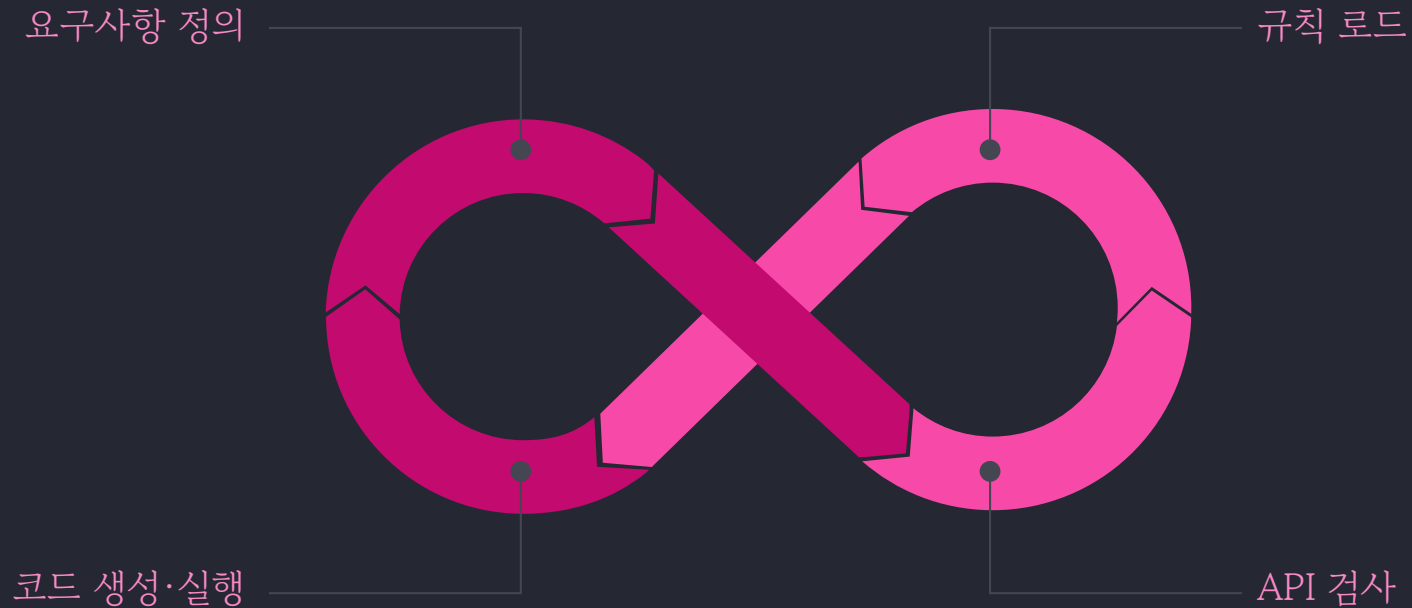
② 검증

HandMirrorMcp, Microsoft Learn, Context7로 API 추측 비용을 줄이고 정확도를 높입니다.

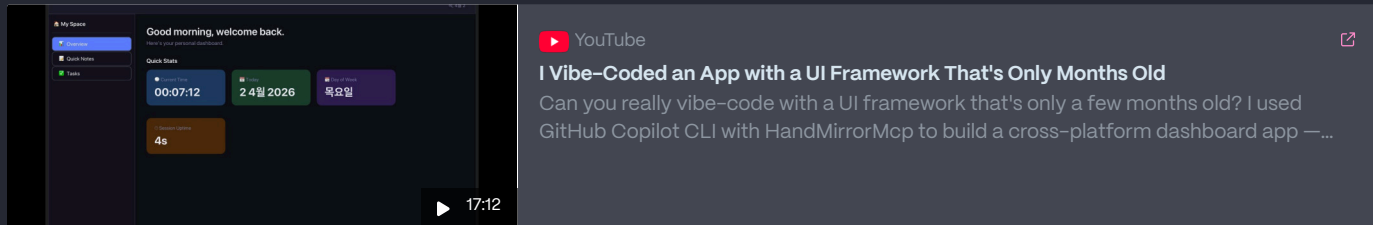
3

③ 실행

`dotnet build` → `dotnet run` → `dotnet publish`로 결과를 직접 확인합니다.



Demonstration



거울이 방금 비춰 준 것

규칙이 강할수록 안정적

Copilot은 제약이 명확할수록 더 일관된 코드를 생성합니다. 빈 규칙 파일은 빈 거울과 같습니다.

MCP가 추측 비용을 낮춤

데스크톱 API를 스스로 추측하는 대신 MCP 도구로 실시간 검사함으로써 오류율이 현저히 줄어듭니다.

build → run 완성의 기준

빌드와 실행 검증까지 거쳐야 비로소 "돌아가는 코드"라고 부를 수 있습니다. 생성만으로는 부족합니다.

GUI 성공 ≠ UX 성공

화면이 뜬다고 완성이 아닙니다. 사용자 흐름과 접근성은 반드시 사람이 직접 확인해야 합니다.



팀에 이 거울을 놓는 방법

01

저장소 템플릿에 함께 넣기

규칙 파일(copilot-instructions.md)과 MCP 설정을 저장소 템플릿에 포함시켜 팀 전체가 동일한 거울을 공유합니다.

02

캘리브레이션 질문으로 시작

첫 프롬프트 전에 짧은 캘리브레이션 질문으로 Copilot이 규칙을 올바르게 인식하고 있는지 확인합니다.

03

build → run 검증 습관화

큰 기능 단위마다 빌드와 실행 검증을 팀의 정식 절차로 정착시킵니다. 자동화 스크립트로 만들면 더 효과적입니다.

04

역할 분리: 사람 vs Copilot

UX·구조 판단은 사람이 리뷰하고, Copilot은 반복 작업과 보일러플레이트 생성을 담당하도록 명확히 분리합니다.



거울이 만능은 아닙니다

AI Agent가 대신할 수 없는 것

- 아키텍처 결정 — 전체 구조를 어떻게 나눌지는 사람의 판단입니다
- UX 완성도 — 실제 사용 흐름의 자연스러움은 사람 눈이 다듬어야 합니다
- 트레이드오프 평가 — 기술 선택의 장단점은 컨텍스트 없이 판단할 수 없습니다

바이브 코딩의 현실적인 목표

바이브 코딩의 목표는 "완성"보다 "학습 속도 향상"에 두는 편이 현실적입니다.

빠른 프로토타이핑, 반복적인 실험, 새로운 API 탐색 — 이 영역에서 Copilot은 강력한 파트너입니다.

하지만 프로덕션 품질의 완성은 여전히 개발자의 책임입니다.

마무리

Key Takeaways

1

거울 없이 에이전트는 눈 뜬 장님

훈련 데이터 밖의 프레임워크에서는 MCP 없는 AI가 추측에 의존하다 실패합니다.

HandMirrorMcp가 그 공백을 채웁니다.

2

제약이 곧 품질이다

copilot-instructions.md로 규칙을 명시하고
build → run 검증 루프를 촘촘히 유지할수록
결과물의 완성도가 높아집니다.

3

바이브 코딩의 주인은 여전히 사람

아키텍처 결정, UX 완성도, 트레이드오프 판단은 AI가 대신할 수 없습니다. Copilot은 강력한 파트너이지, 대체자가 아닙니다.

다시 보기 영상

<https://dotnetuniverse.net/> 에 오시면

오늘 모든 세션의 영상, 발표 자료를

다시 다운로드 받으실 수 있습니다!

고맙습니다!

[linkedin.com/in/rkttu](https://www.linkedin.com/in/rkttu)

